SUMMARY REPORT

CSCI-570 Group 89 Members:

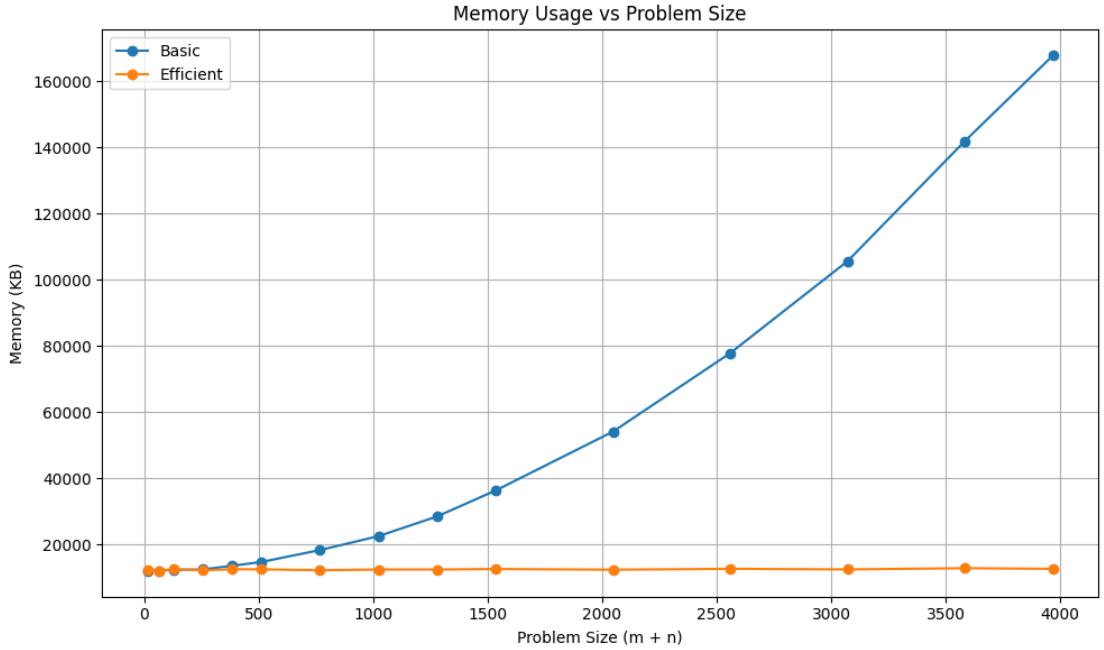
Team Member	USC ID
Gangwani, Sakshi	5476035923
McFry, Adam	8648410555
Patel, Rut	9495840729
Wu, Yuzheng	7516524638
Yajaman, Saatwik	9415230993

Datapoints Output Table

M+N	Time in MS (Basic)	Time in MS (Efficient)	Memory in KB (Basic)	Memory in KB (Efficient)
16	0.023	0.059	15300	15516
64	0.208	0.415	15344	15520
128	0.769	1.425	15460	15480
256	2.904	5.065	15968	15544
384	6.434	11.577	16776	15544
512	13.022	22.341	17980	15548
768	27.300	43.792	21188	15576
1024	48.061	77.692	25684	15600
1280	72.546	120.777	31692	15616
1536	118.049	167.016	39128	15664
2048	195.229	302.717	57212	15656
2560	312.784	480.496	80980	15724
3072	444.687	707.214	108680	15776
3584	613.062	1015.142	144640	15836
3968	754.863	1220.740	170408	15864

Insights

Graph 1 – Memory vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

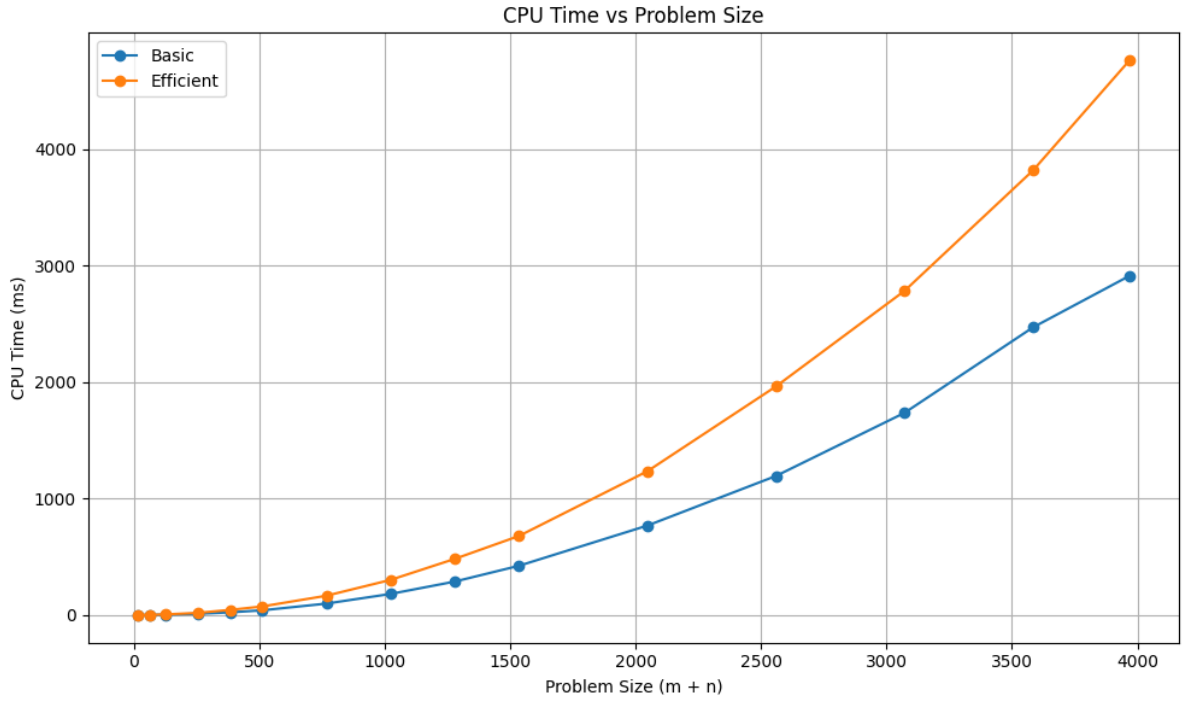
Basic: Polynomial
Efficient: Linear

Analysis of Memory Graph:

The Basic algorithm builds a full 2D grid to store values. Since the grid grows in two dimensions, the memory usage grows quadratically as the input gets larger. This is expected, since the Basic version of the this algorithm uses a 2D table to keep track of all values throughout the duration of the program's execution. This also seems to suggest that the space complexity of the Basic algorithm is O(m*n).

The Efficient algorithm uses a divide-and-conquer strategy that only needs to store two rows of data at any given moment. This means its memory usage grows linearly as the input gets bigger. As expected, this algorithm performs better from a space complexity standpoint, as it is not using the entire 2D matrix that the Basic algorithm uses. The efficient algorithm has a space complexity of O(min(M, N)), since, as mentioned, this implantation only needs a limited amount of memory to store the information for the recursive calls and the current subproblem.

Graph 2 – Time vs Problem Size (M+N)



Nature of the Graph (Logarithmic/ Linear/ Polynomial/ Exponential)

Basic: Polynomial
Efficient: Polynomial

Analysis of CPU Usage Graph:

Unlike the memory usage graph, comparing the CPU time usage between the Basic algorithm and efficient algorithm reveals that both algorithms follow a more similar curve. When using a dynamic programming approach to solving the sequence alignment problem, it's expected that there is a time complexity of O(M*N). For the two strings of size M and N respectively, as the problem size increases, CPU time increases as well; at each step, the solution is based on the calculations and subsequent solutions of other subproblems.

The Efficient algorithm, according to this graph, appears to be using more CPU time than the Basic algorithm. One such explanation for this performance difference is that the Efficient algorithm is using a divide-and-conquer approach in the algorithm; as a result, there are additional compute resources being used in order to calculate the results for merging the subproblems and addressing the recursive calls.

Contribution

5476035923: Equal Contribution
8648410555: Equal Contribution
9495840729: Equal Contribution
7516524638: Equal Contribution
9415230993: Equal Contribution